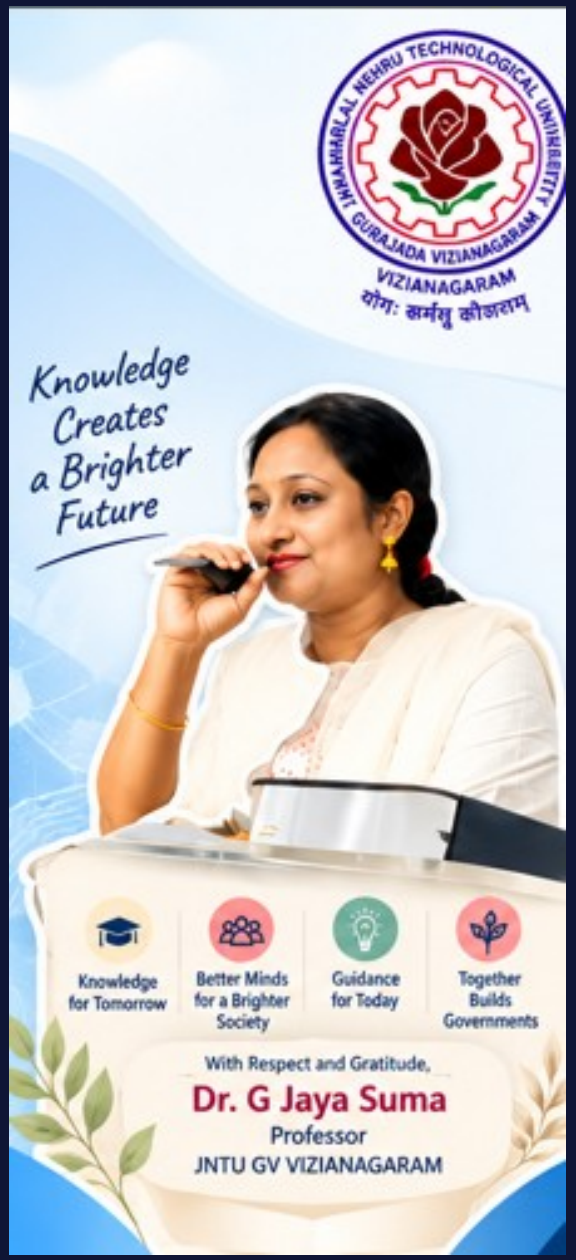


JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Introduction: From Theory to Running Code

Turning the circuits and math from this series into a program you can actually run.

10 MIN | Qiskit | Simulators | Your first circuit

1 What we'll cover in 10 minutes

Two short segments — from the mental shift of coding a circuit to running your first one.

1

5 min

From theory to code: what changes

Circuit diagrams become a handful of function calls — the physics doesn't change.

2

5 min

Your first quantum program

Build, run, and read out a Bell pair — on a simulator, then real hardware.

2 From theory to code: what changes

Circuit diagrams become a handful of function calls — the physics doesn't change.

In simple words

$$|\psi\rangle = \text{circuit} \quad \rightarrow \quad \texttt{qc.h(0); qc.cx(0,1)}$$

Every circuit diagram in this series translates directly into a handful of function calls: one line per gate, one line to measure. The physics doesn't change — only the notation does.

A quantum program is really a recipe: build the circuit, choose a backend, run it, and read out counts instead of a wavefunction.

Q

A quantum SDK

Frameworks like Qiskit, Cirq, and PennyLane let you build circuits in Python, gate by gate.

Σ

Simulators first

A classical simulator runs your circuit exactly, letting you debug logic before touching real hardware.

☁

Real hardware, via the cloud

The same code can target an actual quantum processor over the cloud once your circuit is ready.

3 Your first quantum program

Two qubits, two gates — the simplest circuit that shows genuinely quantum behavior.

A minimal example

H → CNOT → Measure (a Bell pair)

A Hadamard puts qubit 0 into superposition, then a CNOT entangles it with qubit 1 — the smallest circuit that can't be explained without quantum mechanics.

Running it 1,000 times on a simulator should return roughly half ‘00’ and half ‘11’ — never ‘01’ or ‘10’.

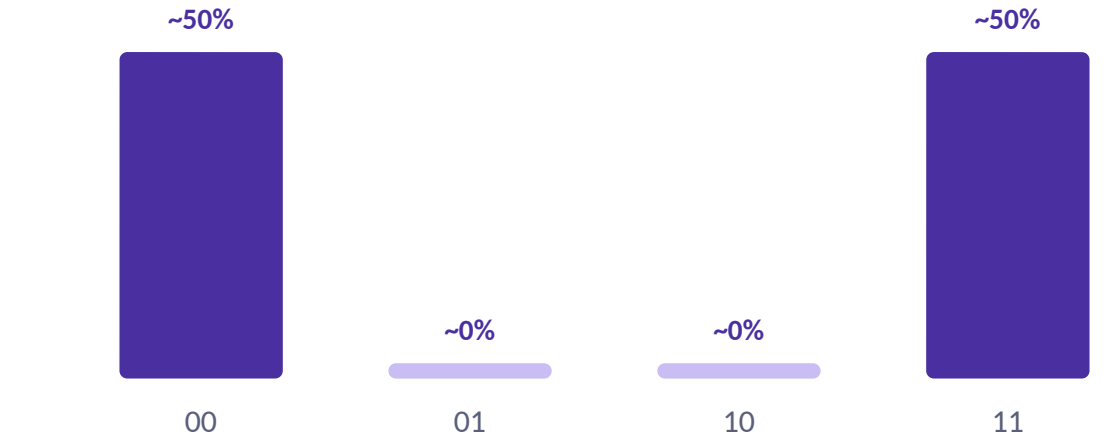
Note: that perfect correlation, with no classical explanation, is the same entanglement running underneath every algorithm in this series.

The code

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])
```

Result on a simulator (1,000 shots)



ONE-LINE TAKEAWAY

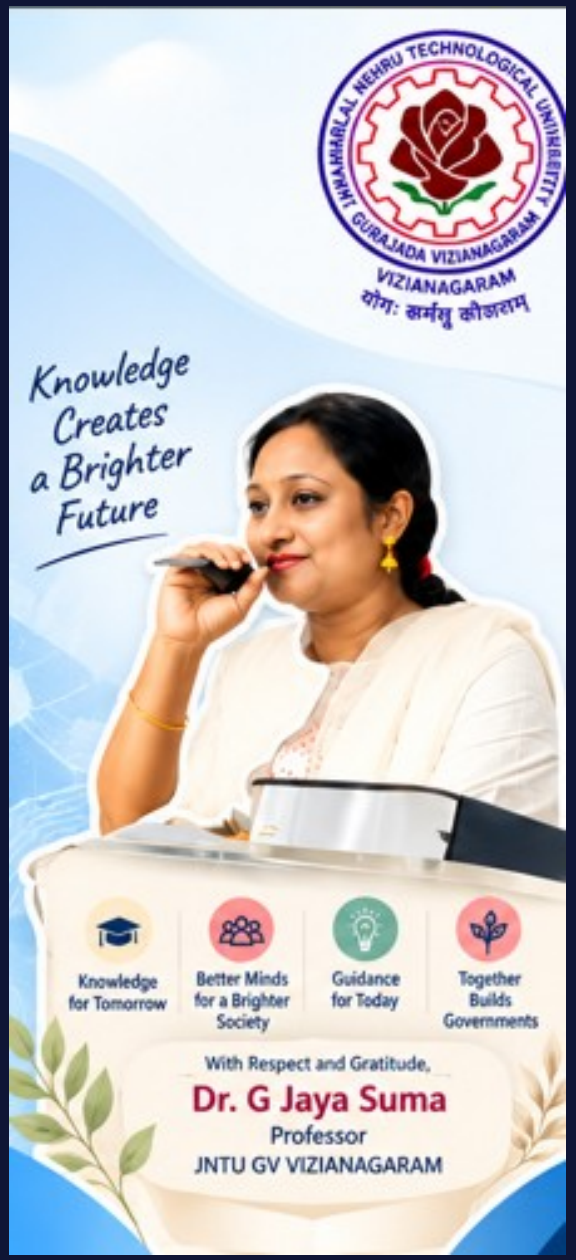
Every gate and circuit in this series is just a few lines of code away — the next step is running it yourself, first on a simulator, then on real quantum hardware.

Qiskit | Simulator → hardware | Bell pair demo

→

Next: Building and Running Your First Circuits

Time to open the notebook: we'll set up Qiskit, build a few more circuits, and run them on real quantum hardware.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

The Quantum SDK Landscape

A tour of the tools that turn quantum circuits into programs you can actually run.

35 MIN | Qiskit | Cirq | PennyLane | Cloud access

1 What we'll cover in 35 minutes

Seven segments, from why SDKs exist to picking the right one for the job.

1 — 5 min

Recap: why we need an SDK

The bridge between gate-level math and a runnable program.

2 — 5 min

The major players

Qiskit, Cirq, PennyLane, and Q# — who backs what.

3 — 5 min

Anatomy of an SDK

Circuits, transpilers, and backends — the pipeline behind every run.

4 — 5 min

Simulators vs. real hardware

Exact but limited, versus scalable but noisy.

5 — 5 min

Cloud platforms

IBM Quantum, Amazon Braket, Azure Quantum, Google Quantum AI.

6 — 5 min

Choosing an SDK

A short checklist for matching the tool to the task.

7 — 5 min

Why it matters: the ecosystem's trajectory

Convergence, open source, and skills that transfer.

2 Recap: why we need an SDK at all

The bridge between the gate-level math we've used and a program that actually runs.

In simple words

circuit diagram → SDK objects → hardware-ready code

An SDK (software development kit) gives you objects for qubits, gates, and measurements, plus tools to compile and send a circuit to a backend — turning the diagrams from this series into a real program.

Without an SDK, you'd be hand-writing pulse-level instrument commands for one specific machine; the SDK abstracts that away.

C

Circuit construction

Build circuits programmatically, gate by gate — the same way we've drawn them on slides.

⇆

Backend abstraction

The same circuit can target a simulator or dozens of real devices by changing one line.

T

Compilation ('transpiling')

The SDK rewrites your circuit into the gate set and qubit layout a target device actually supports.

3 The major players

Four SDKs cover most of what you'll encounter — same core ideas, different syntax.

Backer, language, and best-fit use case

Qiskit	Cirq	PennyLane	Q#
Backed by	**Backed by**	**Backed by**	**Backed by**
IBM	Google	Xanadu	Microsoft
Language	**Language**	**Language**	**Language**
Python	Python	Python	Q# / .NET
Best for	**Best for**	**Best for**	**Best for**
Circuit-level control & IBM hardware access	NISQ research & Google hardware	Quantum machine learning, differentiable circuits	Large-scale algorithms & Azure Quantum

4 Anatomy of a quantum SDK

Three layers sit between the circuit you write and the answer you get back.

The three layers

Circuit → Transpiler → Backend

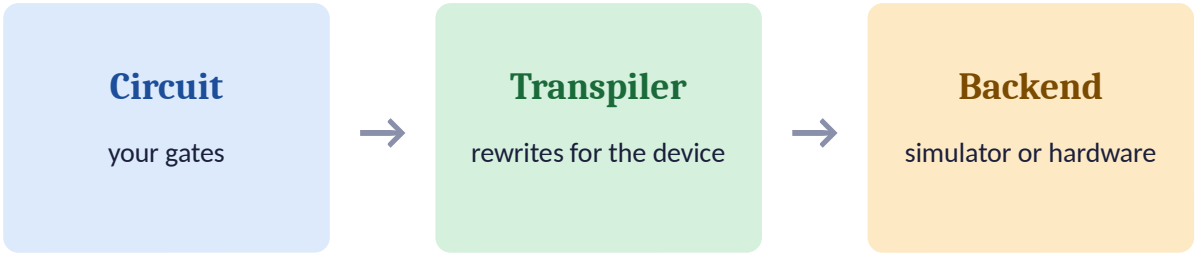
1. Circuit — the abstract sequence of gates you write, hardware-agnostic.

2. Transpiler — rewrites it using only the gates and qubit connections a specific device supports, trying to minimize depth and error.

3. Backend — where it actually executes: a simulator, or a real quantum processor.

Note: the same circuit can transpile very differently for two devices with different native gate sets — that's why runtime and fidelity vary by backend.

The pipeline



Every SDK call you write ultimately walks this same pipeline, whatever the syntax looks like.

5 Simulators vs. real hardware

Two very different backends behind the same line of code.

Two very different backends

Simulator: exact but exponential. Hardware: real but noisy.

A classical simulator computes the exact quantum state — perfect for debugging small circuits, but the memory it needs doubles with every added qubit.

Real hardware scales in qubit count but introduces noise: decoherence, gate errors, and readout errors a simulator doesn't have.

Careful: a circuit that works perfectly on a simulator can fail completely on real hardware if it's too deep for the device's coherence time.

	Simulator	Real Hardware
Qubit count (practical)	~30–40	100s–1,000s
Result	Exact	Noisy / probabilistic
Per-gate fidelity	100%	~99–99.9%
Cost scaling	Exponential in qubits	Roughly fixed per shot

6 Cloud platforms

You don't need a quantum computer on your desk — these run your circuit remotely.

Hardware access, free tier, and standout feature

IBM Quantum	Amazon Braket	Azure Quantum	Google Quantum AI
Hardware	**Hardware**	**Hardware**	**Hardware**
IBM superconducting qubits	IonQ, Rigetti, QuEra & more	Quantinuum & other partners	Google superconducting qubits
Access	**Access**	**Access**	**Access**
Generous free tier via Qiskit	Pay-per-shot, no free hardware tier	Integrated with Q# / Qiskit, research credits	Mostly research partnerships

7 Choosing an SDK: criteria that matter

A short checklist beats a long debate — most choices come down to three questions.

A simple decision checklist

Target hardware: if you need a specific vendor's device, start with the SDK that vendor backs.

Task type: circuit-level algorithm design, hybrid variational workflows, and quantum ML each favor a different SDK's strengths.

Community, documentation, and how actively the project ships all matter more once you're past the first tutorial.

Want IBM hardware & a big community?

Qiskit — the largest ecosystem, extensive docs, and a generous free tier.

Doing quantum machine learning?

PennyLane — built around differentiable circuits that plug into PyTorch and TensorFlow.

Building enterprise-scale, hybrid apps?

Q# — designed for large algorithms and tight Azure Quantum integration.

8 Why it matters: the ecosystem's trajectory

Different syntax, same ideas — and the gap between SDKs is shrinking.

The big picture

Every SDK converges on: circuit, transpiler, backend

Despite different syntax, all the major SDKs converge on the same core abstractions. Learning one makes picking up the others mostly a matter of new method names, not new concepts.

Interoperability layers let circuits move between frameworks and hardware providers.

Note: OpenQASM (Open Quantum Assembly Language) acts as a common exchange format between many SDKs and hardware providers.

⇆ Interoperability is growing

OpenQASM and similar formats let circuits move between SDKs and hardware providers.

⌘ Open source, fast moving

All major SDKs are open source and evolve monthly — expect breaking changes as you learn.

✓ The skill transfers

Circuits, transpilation, and backends matter more than any one SDK's particular syntax.

ONE-LINE TAKEAWAY

Every major SDK — Qiskit, Cirq, PennyLane, Q# — wraps the same circuit-transpiler-backend pipeline in different syntax; learn the pattern once, and the rest follow easily.

Circuit → Transpiler → Backend

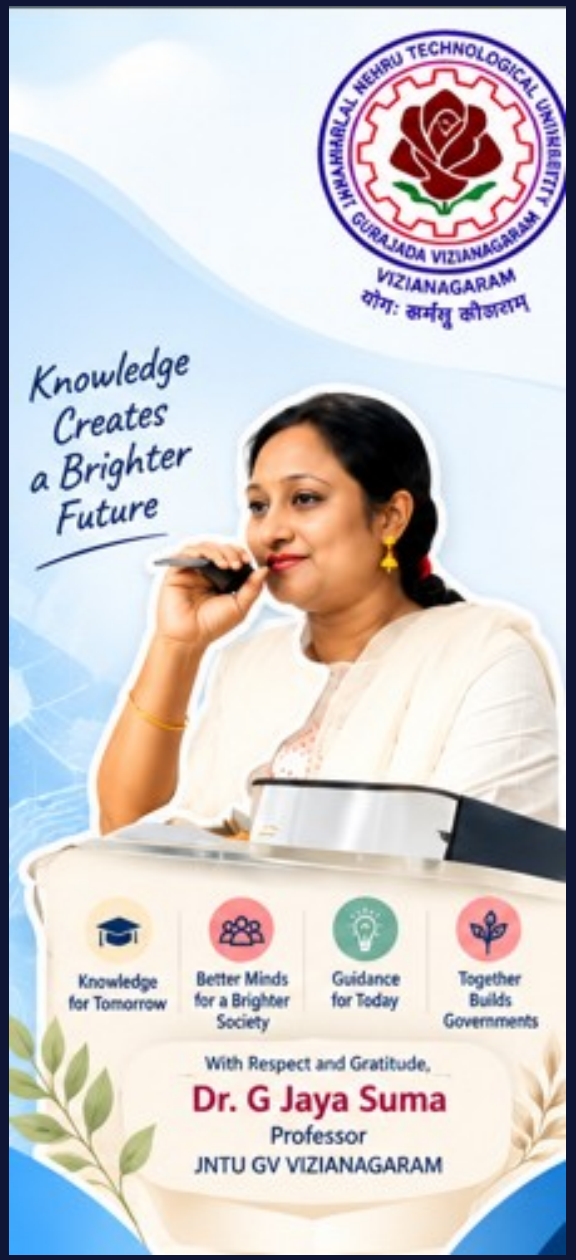
Open source & fast-moving

OpenQASM interoperability

→

Next: Building and Running Your First Circuits

With the landscape mapped, it's time to open an SDK and write real circuits, gate by gate.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Hands-On Qiskit Fundamentals

From pip install to your first jobs on real quantum hardware.

50 MIN | Qiskit | Circuits & gates | Simulators | Real hardware

1 What we'll cover in 50 minutes

Seven segments, from pip install to jobs running on a real quantum chip.

1 — 5 min

Setting up: installing Qiskit

One pip install gets you a circuit builder and a local simulator.

2 — 5 min

Building circuits: qubits, gates, registers

QuantumCircuit, method-call gates, and Qiskit's drawing conventions.

3 — 10 min

Running on a simulator

The Sampler primitive vs. the exact Statevector simulator.

4 — 10 min

Visualizing results

Histograms, Bloch spheres, and circuit diagrams straight from code.

5 — 5 min

Parameterized circuits & loops

Placeholder angles and Python loops for repeated structure.

6 — 10 min

Worked example: Bell & GHZ states

Build and run two entangled states with the same few lines.

7 — 5 min

Running on real hardware

Swapping the simulator for an IBM Quantum device.

2 Setting up: installing Qiskit

One pip install gets you a circuit builder and a local simulator.

In simple words

pip install qiskit qiskit-aer

Qiskit installs like any Python package. The core package gives you circuit-building tools; qiskit-aer adds the local simulator you'll use for most of this session.

A virtual environment keeps Qiskit's dependencies separate from the rest of your Python setup — recommended, but not required to follow along.

One pip install

pip install qiskit qiskit-aer pulls in the circuit library and a local simulator in one step.

Works anywhere

Runs in a plain script, a Jupyter notebook, or Google Colab — no special hardware needed to start.

Free IBM account (optional)

Sign up at quantum.ibm.com to eventually run circuits on real hardware, at no cost.

3 Building circuits: qubits, gates, registers

A circuit is a Python object — gates are just method calls on it.

The building blocks

QuantumCircuit(qubits, bits)

A QuantumCircuit is built from a QuantumRegister (qubits) and a ClassicalRegister (bits to store results) — Qiskit creates both automatically when you pass plain numbers.

Gates are added as method calls: .h(), .x(), .cx(), .rz(θ), and dozens more, applied to one or more qubit indices.

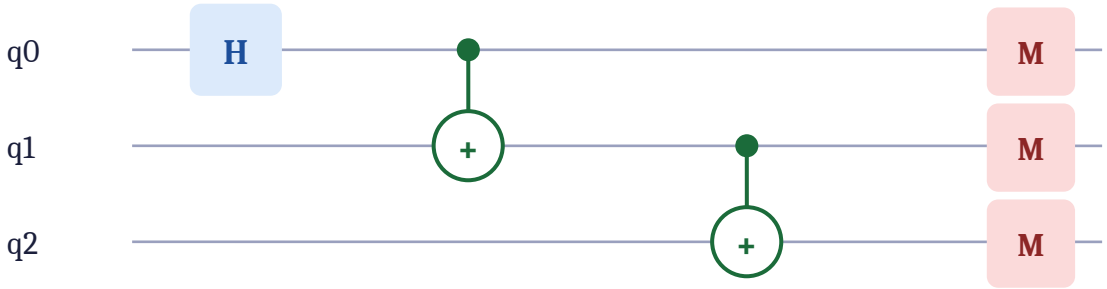
Note: Qiskit numbers qubits and bits from 0, and draws circuits left-to-right in time order — the same convention used throughout this series.

The code

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(3, 3)
qc.h(0)
qc.cx(0, 1)
qc.cx(1, 2)
qc.measure([0,1,2], [0,1,2])
```

What it draws



4 Running on a simulator

Two ways to simulate — many noisy samples, or one exact answer.

Two ways to simulate

AerSimulator() → backend.run(circuit)

The Sampler primitive runs your circuit many times (‘shots’) and returns measurement counts — mimicking real hardware.

The Statevector simulator instead returns the exact final quantum state — no shots, no randomness — useful for checking your circuit does what you think.

Note: real hardware can only ever give you samples, never the exact statevector — that's a diagnostic tool unique to simulation.

The code

```
from qiskit_aer import AerSimulator
from qiskit.primitives import Sampler

sampler = Sampler()
result = sampler.run(qc, shots=1000).result()
counts = result.quasi_dists[0]
```

Sampler vs. Statevector

Output	Measurement counts	Full amplitude vector
Randomness	Yes — shot noise	None — exact
Matches hardware?	Yes, in spirit	No — sim-only tool

5 Visualizing results

Turning raw numbers into pictures you can actually read.

Seeing what happened

plot_histogram(counts)

Qiskit's visualization module turns raw counts into a bar chart, a state vector into Bloch-sphere arrows, and a circuit into the diagrams used throughout this series.

These aren't just pretty pictures — a histogram instantly tells you whether your circuit produced the correlation you expected.

Note: plot_histogram and plot_bloch_multivector need matplotlib installed — pip install matplotlib if you see an import error.

plot_histogram()

Bar chart of measurement counts — your first check that entanglement or interference worked.

plot_bloch_multivector()

Each single qubit's state as an arrow on a sphere — intuitive, but clean only for unentangled qubits.

circuit.draw()

Renders the exact circuit-diagram style used throughout this series, straight from your code.

6 Parameterized circuits & loops

Templates you can reuse, instead of hand-writing every gate.

Circuits that take inputs

Parameter("θ") → qc.rz(theta, 0)

A Parameter is a placeholder angle you bind to a real number later — essential for variational algorithms where a classical optimizer repeatedly adjusts circuit angles.

A Python for-loop builds repeated structure — like R Grover iterations — without hand-writing every gate.

Note: binding a parameter with assign_parameters(...) creates a new circuit — the original template stays reusable.

The code

```
from qiskit.circuit import Parameter

theta = Parameter("θ")
qc = QuantumCircuit(1)
qc.rx(theta, 0)
bound = qc.assign_parameters({theta: 1.57})
```

Looping instead of repeating gates by hand

```
for _ in range(3):
    qc.h(0)
    qc.z(0)
```

7 Worked example: Bell & GHZ states

Build and run two entangled states with the same few lines.

Two circuits, one recipe

Bell state (2 qubits): H(0) then CX(0,1) → outputs only ‘00’ and ‘11’, roughly 50/50.

GHZ state (3 qubits): H(0), CX(0,1), CX(1,2) → outputs only ‘000’ and ‘111’ — entanglement across all three qubits at once.

Both circuits run in under a millisecond on the simulator; the real test is running them on IBM hardware and seeing how close the counts stay to ideal.

Same code shape

Both states use the identical build → run → plot_histogram pattern from this segment.

More qubits, same trick

Chaining CX gates from one qubit to the next generalizes a Bell pair to any number of qubits.

Hardware will disagree slightly

Expect a small fraction of stray results on real hardware — that's noise, not a bug.

8 Running on real hardware

Swapping the simulator for an IBM Quantum device is almost a one-line change.

From simulator to a real chip

service.least_busy() → backend.run(...)

Authenticate once with your IBM Quantum API token, then pick a device — swapping the backend is almost a one-line change from the simulator code you've already written.

Real jobs queue behind other users' jobs — expect anywhere from seconds to hours of wait, depending on how busy the chosen device is.

Note: your circuit is transpiled for the specific device automatically — the gate sequence on hardware may look different from what you wrote.

The code

```
from qiskit_ibm_runtime import
QiskitRuntimeService


service = QiskitRuntimeService()

backend = service.least_busy(operational=True)
job = backend.run(transpile(qc, backend),
shots=1000)
```

Free plan: IBM's open plan gives everyone a monthly allotment of real-hardware runtime — plenty to reproduce every example in this session.

ONE-LINE TAKEAWAY

Every Qiskit program follows the same shape — build a circuit, choose a backend, run it, and visualize the counts — whether that backend is your laptop or a real quantum chip.

Build → Run → Visualize

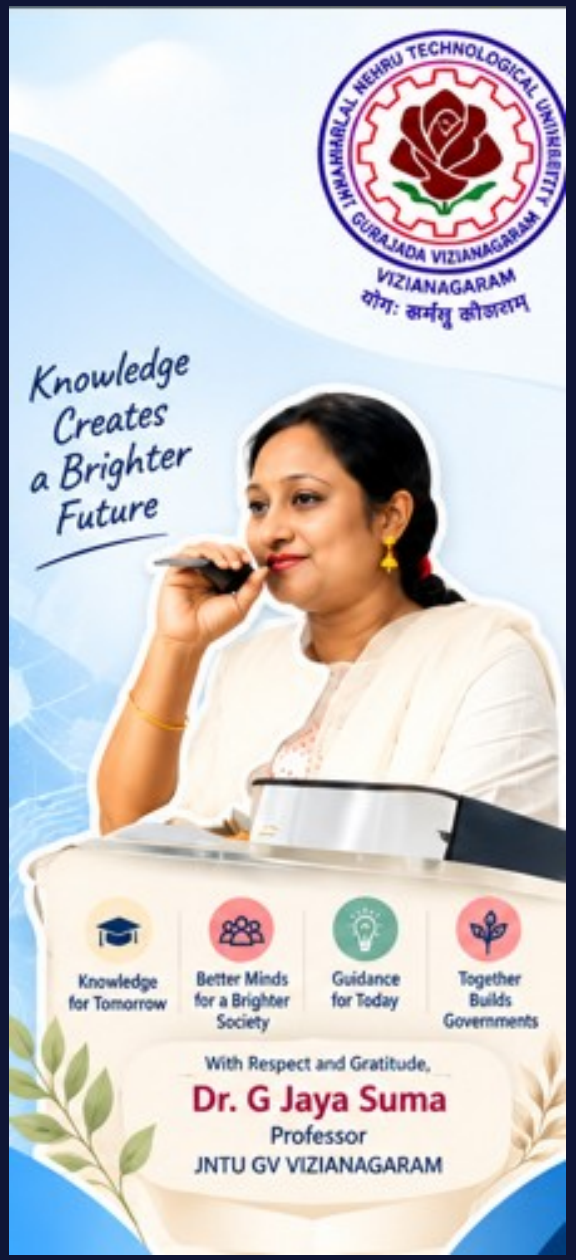
Simulator-first workflow

One-line switch to hardware

→

Next: Variational Algorithms — VQE and QAOA in Practice

With the fundamentals down, we'll combine circuits with a classical optimizer to tackle real optimization and chemistry problems.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Simulators vs Real Hardware (Cloud QPUs)

What actually runs your circuit — and why the answer changes what you can trust.

40 MIN | Statevector | Noise models | Cloud QPUs | NISQ

1 What we'll cover in 40 minutes

Seven segments, from what's inside each backend to closing the sim-to-real gap.

1 — 5 min

Recap: why the distinction matters

Same circuit, same code — very different answers.

2 — 5 min

Inside a classical simulator

Statevector, density-matrix, and MPS methods, and their limits.

3 — 5 min

Inside a real QPU

Qubits, control electronics, and the cryostat that keeps it all cold.

4 — 10 min

Noise: where it comes from

Decoherence, gate errors, and readout errors, and how they compound.

5 — 5 min

Noisy simulation: a middle ground

Testing against realistic noise before you touch a queue.

6 — 5 min

Choosing a backend

A practical decision path for picking the right tool.

7 — 5 min

Why it matters: the sim-to-real gap

The central challenge of today's NISQ era.

2 Recap: why the distinction matters

The same circuit, the same code — but the backend decides what answer you actually see.

In simple words

same circuit → simulator (exact) or QPU (noisy)

Every circuit we've built runs the same on paper, but the backend you pick decides whether you see the ideal answer or a noisy approximation of it.

Simulators are a debugging tool; real hardware is the actual experiment — mixing them up leads to false confidence.

=

Same code, different backend

Swapping AerSimulator() for a real device is one line — but the results can look very different.

✓

Simulators: ground truth

No noise, no queue, unlimited runs — perfect for checking your logic is correct.

⚡

Hardware: the real test

Only real hardware tells you whether an algorithm survives contact with physical qubits.

3 Inside a classical simulator

Three methods, three trade-offs between accuracy, noise, and how many qubits you can fit.

Three simulation methods

Statevector: 2^n complex amplitudes, stored exactly

Statevector simulation stores every amplitude exactly — perfect accuracy, but memory doubles with each added qubit, capping out around 30–40 qubits on a powerful machine.

Density-matrix simulation adds noise modeling at the cost of squaring the memory again; matrix-product-state (MPS) simulation trades exactness for handling more qubits when entanglement stays limited.

Note: no classical simulator scales to hundreds of qubits for a general circuit — that's precisely the wall real hardware is built to get around.

Three simulator backends

Statevector	Density Matrix	MPS
Memory	**Memory**	**Memory**
Exponential (2^n)	Exponential (4^n)	Depends on entanglement
Models noise?	**Models noise?**	**Models noise?**
No	Yes	Limited
Qubit limit	**Qubit limit**	**Qubit limit**
~30–40 qubits	~15–20 qubits	100s (low entanglement)

4 Inside a real QPU

The qubits are the easy part to draw — keeping them alive is the hard engineering.

What's actually there

qubits + control electronics + cryostat

A superconducting QPU keeps its qubits near absolute zero (~15 millikelvin) inside a dilution refrigerator, with microwave pulses from room-temperature electronics driving every gate and measurement.

Other qubit technologies — trapped ions, neutral atoms, photonics — swap the hardware but face the same core problem: keeping a fragile quantum state alive long enough to compute.

Note: the wiring and control electronics — not the qubits themselves — are often the harder engineering problem at scale.

Superconducting qubits

IBM, Google: fast gates, but need dilution-fridge cooling down near 15 millikelvin.

Trapped ions

IonQ, Quantinuum: very high fidelity, but slower gate times than superconducting chips.

Neutral atoms / photonics

QuEra, Xanadu: newer approaches with different, still-evolving scaling trade-offs.

5 Noise: where it comes from

Three sources of error stack up on every real circuit you run.

Four ways things go wrong

T1 (energy decay), T2 (dephasing)

Every qubit slowly loses its state through decoherence, measured by two timescales: T1 (how long before it decays to $|0\rangle$) and T2 (how long its phase stays coherent) — typically tens to hundreds of microseconds today.

Gate and readout errors add on top: a two-qubit gate might fail ~0.5–1% of the time, and measurement itself can misread a qubit a few percent of the time.

Careful: errors compound — a 100-gate circuit at 99% average gate fidelity keeps well under half its signal by the end.

T

Decoherence (T1 / T2)

The qubit's memory fades on its own, whether or not you touch it.

G

Gate errors

Every physical operation is imperfect — errors accumulate with circuit depth.

R

Readout errors

Measuring a qubit can itself misreport its state a few percent of the time.

6 Noisy simulation: a middle ground

See roughly what real hardware would give you, without touching a queue.

Testing before you pay

AerSimulator(noise_model=...)

A noise model injects realistic T1/T2, gate, and readout errors into an ordinary simulator run — you see roughly what real hardware would give you, without touching a queue.

Many providers publish a live calibration snapshot of their actual devices, so you can simulate against today's real noise profile.

Note: a noisy simulator is still a model — it won't capture effects like crosstalk between distant qubits as accurately as the real chip.

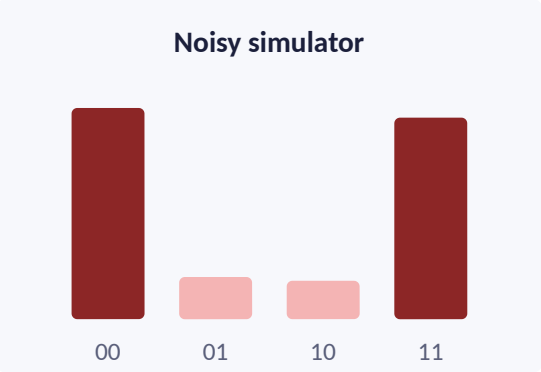
The code

```
from qiskit_aer.noise import NoiseModel
from qiskit_ibm_runtime.fake_provider import
FakeSherbrooke

noise_model =
NoiseModel.from_backend(FakeSherbrooke())
noisy_sim = AerSimulator(noise_model=noise_model)
```

Bell state: ideal vs. noisy





7 Choosing a backend: a practical guide

Three questions get you to the right tool almost every time.

A simple decision path

Debugging circuit logic? → exact Statevector simulator — no shots needed, instant, free.

Estimating real-world performance? → noisy simulator with a live calibration snapshot.

Need the actual answer for a paper, demo, or attack? → real QPU, chosen by lowest queue and best fidelity for your circuit.

Small circuit, need certainty

Statevector simulator — exact, instant, and free.

Medium circuit, need realism

Noisy simulator with a fake-backend noise model.

It has to be real

Real QPU via the cloud — budget time for the queue.

8 Why it matters: the sim-to-real gap

The central challenge standing between today's NISQ devices and the fault-tolerant future.

The gap that matters

ideal fidelity 100% → real fidelity often <50%

A circuit that looks perfect on paper and in an ideal simulator can lose most of its signal to noise by the time it runs as a 100+ gate circuit on real hardware today.

This gap is exactly why error correction, better qubits, and noise-aware compilation are such active research areas.

Note: 'NISQ' — noisy intermediate-scale quantum — is the name for exactly this era: real hardware that is genuinely quantum, but noisy and uncorrected.

↑

Simulators set the ceiling

They show what's possible if noise were solved — a target, not a promise.

↓

Hardware shows the floor

Real devices reveal today's actual, noisy performance.

Δ

The gap is the research problem

Closing it is exactly what quantum error correction and better qubits are for.

ONE-LINE TAKEAWAY

A simulator tells you what your circuit should do; real hardware tells you what it actually does today — and the gap between the two is the central challenge of the NISQ era.

Exact vs. noisy

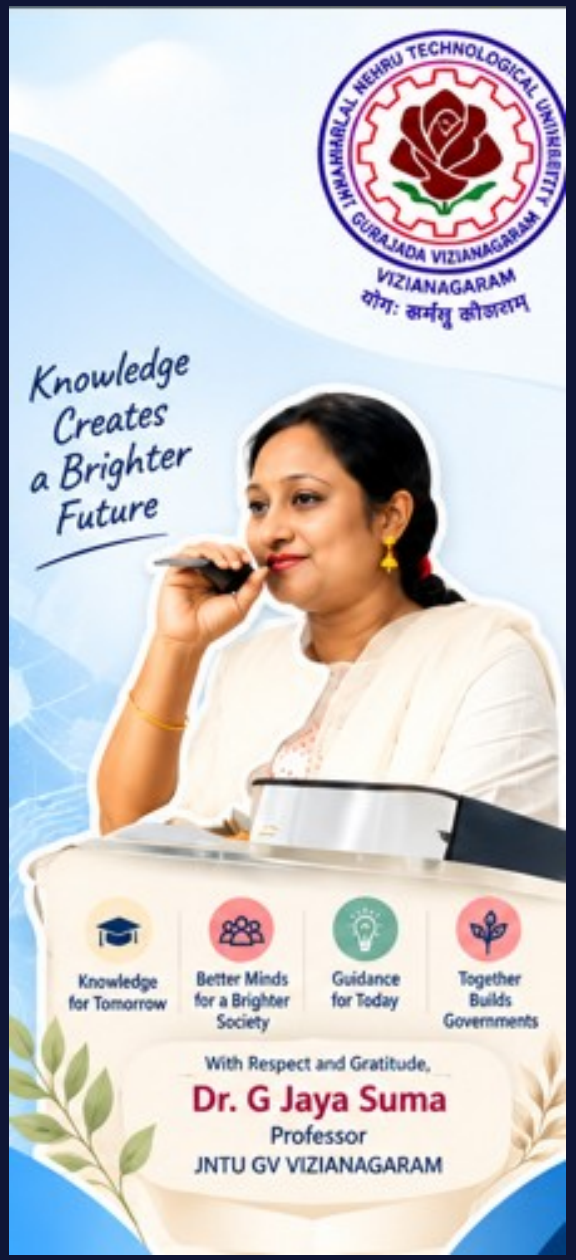
Noise models bridge the gap

NISQ reality

→

Next: Quantum Error Correction — The Path to Fault Tolerance

We've seen where noise comes from — next we look at how to fight back and build reliable logical qubits from noisy physical ones.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Noise and Decoherence

Why quantum states don't stay quantum for long — and how to measure it.

45 MIN | T1 & T2 | Density matrix | Decoherence | Error budgets

1 What we'll cover in 45 minutes

Seven segments, from the Bloch-sphere picture to turning coherence times into a budget.

1 — 5 min

Recap: why noise matters

No qubit is ever perfectly isolated from its environment.

2 — 5 min

The Bloch sphere picture of decoherence

Watching a state's arrow shrink toward the center.

3 — 10 min

T1 relaxation: energy decay

How long before an excited qubit falls back to $|0\rangle$.

4 — 10 min

T2 dephasing: losing phase

How long the relative phase between $|0\rangle$ and $|1\rangle$ survives.

5 — 5 min

Gate and readout errors

Imperfect control and measurement, beyond decoherence.

6 — 5 min

Modeling noise: the density matrix

Why mixed states need more than a wavefunction.

7 — 5 min

Why it matters: coherence budgets

Turning T1 and T2 into a hard limit on circuit depth.

2 Recap: why noise matters

No qubit is ever perfectly isolated from the world around it.

In simple words

$|\psi\rangle$ (pure) → ρ (mixed) as noise creeps in

Every real qubit is coupled, even weakly, to its environment — stray photons, thermal vibrations, imperfect control pulses. That coupling leaks information out of the qubit and randomness in.

Decoherence is the umbrella term for this: the gradual loss of the clean quantum superposition every algorithm in this series depends on.

⚠️

Not a bug, a physical fact

No isolation is perfect; some coupling to the environment is unavoidable above absolute zero.

2

Two distinct effects

Energy relaxation (T1) and dephasing (T2) degrade a qubit in different ways, on different timescales.

!

The enemy of every algorithm

Grover, Shor, and every circuit in this series assume noise-free qubits — real noise erodes that assumption.

3 The Bloch sphere picture of decoherence

A pure state is a full-length arrow — noise shrinks it toward the center.

Watching the arrow shrink

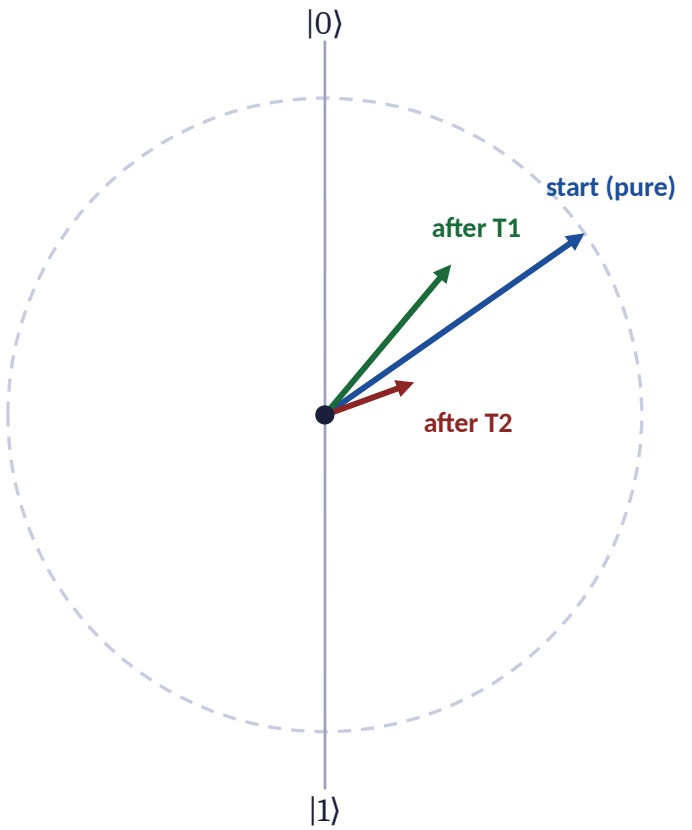
$|\psi\rangle$ → Bloch vector shrinks toward the center

A pure qubit state is a unit-length arrow on the Bloch sphere. As noise acts, that arrow shrinks toward the center — a shorter arrow means a more mixed, less certain state.

A fully decohered qubit sits at the exact center: maximum uncertainty, no quantum information left.

Note: relaxation pulls the arrow toward the pole ($|0\rangle$); dephasing shrinks it toward the axis without changing height — the two act geometrically differently.

The shrinking Bloch vector



Center = maximally mixed: every measurement outcome equally likely.

4 T1 relaxation: energy decay

How long an excited qubit waits before it falls back to its ground state.

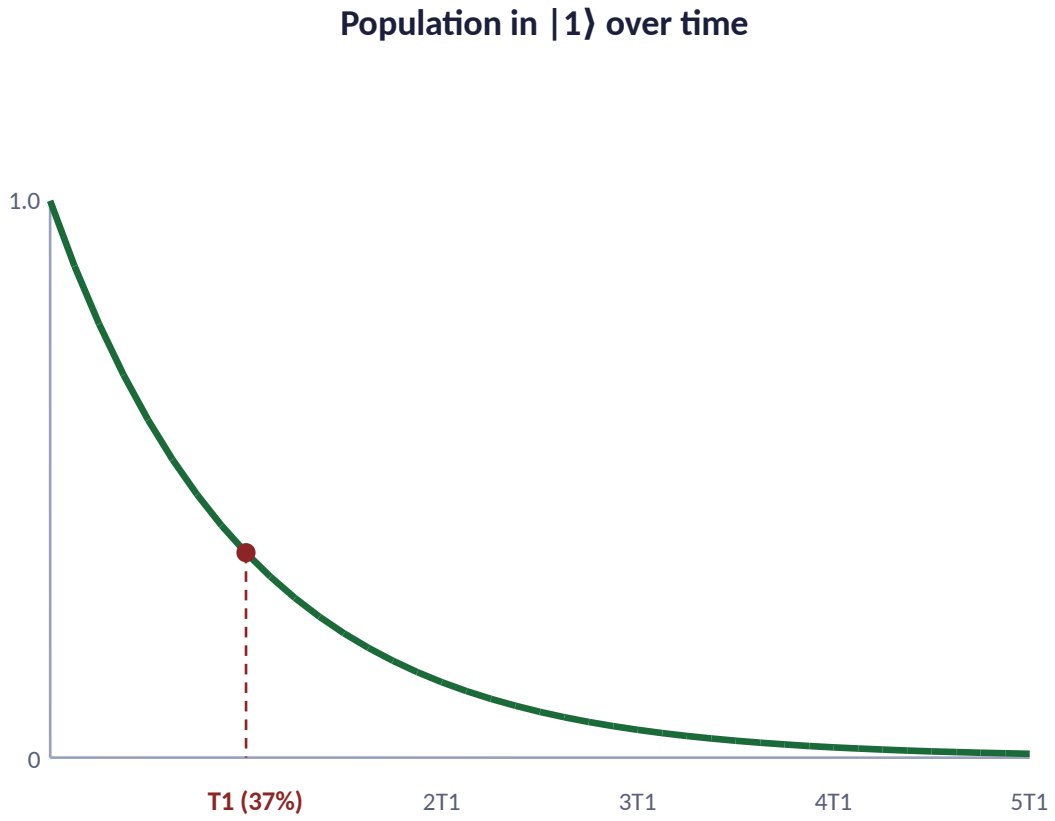
Losing energy to the environment

$$P(|1\rangle, t) = P_0 \cdot e^{(-t / T1)}$$

T1 is the characteristic time for an excited qubit $|1\rangle$ to spontaneously decay to $|0\rangle$, releasing its energy to the environment — much like an atom emitting a photon.

Today's superconducting qubits typically reach T1 of 50–300 microseconds; trapped ions can hold state for many seconds.

Careful: T1 sets a hard ceiling on circuit runtime — signal decays exponentially, not linearly, so doubling circuit depth costs far more than doubling T1 buys back.



5 T2 dephasing: losing phase

A qubit can keep its energy but still lose the phase that makes it quantum.

Losing the phase, not the energy

$$\text{coherence} \propto e^{(-t / T2)}, \quad T2 \leq 2 \cdot T1$$

T2 measures how long the relative phase between $|0\rangle$ and $|1\rangle$ stays well-defined — a qubit can dephase completely while keeping the 'right' energy.

T2 can never exceed 2×T1, and extra noise sources — like magnetic field fluctuations — often make it noticeably shorter than that bound in practice.

Note: a Ramsey or spin-echo experiment is the standard way to measure T2 directly, by watching an equatorial Bloch vector lose its way around the sphere.

Typical T1 and T2 by qubit platform

Superconducting	Trapped Ion	Neutral Atom
T1	T1	T1
~100–300 µs	seconds+	~1–10 s
T2	T2	T2
~100–200 µs	~0.5–1 s	~1 s

6 Gate and readout errors

Even a perfectly isolated qubit would still suffer from imperfect control.

Errors beyond decoherence

$$\text{gate fidelity } F, \text{ readout fidelity } R < 100\%$$

Even a qubit with infinite T1/T2 would still suffer from imperfect control: every physical gate pulse has some miscalibration, and every measurement can misreport the state.

These errors don't accumulate like decoherence — they occur once per operation, so deeper circuits with more gates rack up proportionally more error, regardless of how long they take.

Note: today's best two-qubit gates reach ~99.5–99.9% fidelity; single-qubit gates are usually 10× better still.

C

Coherent errors

Systematic miscalibration — like a slightly wrong rotation angle — that adds up predictably.

R

Incoherent errors

Random fluctuations that behave like extra decoherence and are harder to correct for.

M

Readout errors

Measuring $|0\rangle$ as $|1\rangle$ (or vice versa) a few percent of the time, regardless of decoherence.

7 Modeling noise: the density matrix

A single wavefunction can't describe a mixed state — we need one more layer of math.

Beyond the wavefunction

$$\rho = \Sigma_i \, p_i \, |\psi_i\rangle\langle\psi_i|$$

A single wavefunction $|\psi\rangle$ can only describe a pure state. Once noise mixes in classical uncertainty, we need the density matrix ρ — a weighted mix of possible pure states.

Diagonal entries of ρ are ordinary probabilities; off-diagonal entries capture coherence — exactly what dephasing destroys.

Note: Kraus operators describe how a noisy channel acts on ρ — the same mathematical object Qiskit's NoiseModel builds under the hood.

The off-diagonal terms vanish as T2 runs out



Orange cells hold coherence — they fade to gray as T2 runs out; the diagonal (teal) probabilities are untouched.

8 Why it matters: coherence budgets

T1 and T2 turn into a hard limit on how much a circuit can do.

Turning T1 / T2 into a budget

$$\text{circuit depth} \times \text{gate time} \ll T1, T2$$

Every circuit has an implicit time budget: total run time must stay well under the qubits' T1 and T2, or the signal decoheres before you even measure.

This is why NISQ-era algorithms favor short, shallow circuits, and why longer algorithms like Shor's need error correction rather than just ‘better’ qubits.

Note: as gate times shrink and T1/T2 grow, the number of operations you can run before decoherence dominates keeps climbing — this ratio is the circuit's ‘error budget.’

△

Shallow circuits survive

Variational algorithms (VQE, QAOA) are popular partly because they fit within today's coherence budgets.

∞

Longer algorithms need correction

Shor's algorithm at cryptographic scale needs far more operations than any current T1/T2 allows uncorrected.

Q

The metric to watch

Quantum volume and similar benchmarks combine qubit count, connectivity, and error rates into one number.

ONE-LINE TAKEAWAY

T1 tells you how long a qubit keeps its energy, T2 tells you how long it keeps its phase — and every circuit you design has to finish well before either runs out.

T1: energy decay

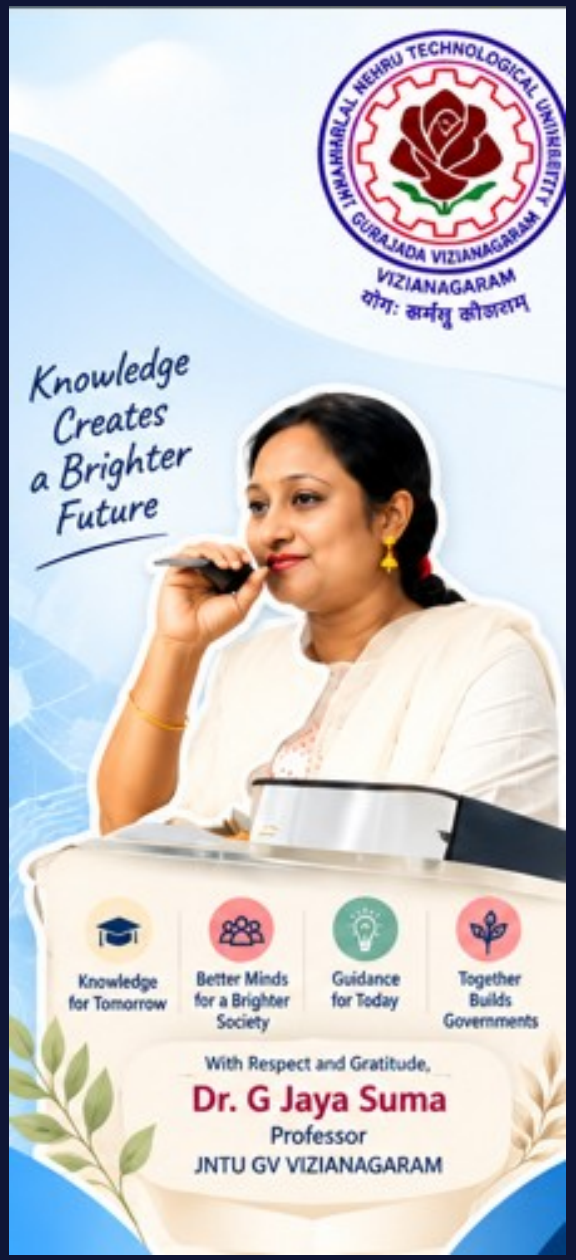
T2: phase decay

Error budgets

→

Next: Quantum Error Correction — The Path to Fault Tolerance

Now that we know exactly how and why qubits fail, we can look at how to build reliable logical qubits out of noisy physical ones.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Noise Mitigation and Circuit Compilation

Squeezing more signal out of noisy hardware — before error correction arrives.

35 MIN | Transpilation | Qubit routing | ZNE | Readout calibration

1 What we'll cover in 35 minutes

Seven segments, from compiling smarter to squeezing a cleaner answer out of noisy runs.

1 — 5 min

Recap: why mitigation matters now

Full error correction is years away — these tools help today.

2 — 5 min

Circuit compilation: abstract to native

Turning H, CX, and friends into hardware-native gates.

3 — 5 min

Optimization passes

Cancellation, commutation, and scheduling shorten a circuit.

4 — 5 min

Qubit routing & mapping

Working around limited connectivity with SWAP gates.

5 — 5 min

Zero-noise extrapolation

Deliberately adding noise to extrapolate back to zero.

6 — 5 min

Readout error mitigation

Calibrating and correcting the last, noisiest step.

7 — 5 min

Why it matters: mitigation vs. correction

What's practical today, and what only scales with qubits.

2 Recap: why mitigation matters now

Full error correction is years away — these tools help on today's hardware.

In simple words

raw noisy result → mitigated estimate

Full quantum error correction needs far more qubits than today's devices have. Until then, two practical strategies squeeze more useful signal out of noisy hardware: compiling circuits smarter, and mitigating errors after the fact.

Neither approach removes noise — they reduce its damage or correct for its average effect, without the overhead of full error correction.

C

Compilation: fewer, better gates

Rewriting a circuit to use fewer operations, matched to what the hardware actually does well.

M

Mitigation: correcting the average

Statistical techniques that cancel out noise's average effect across many runs.

≠

Not a replacement for correction

Both buy time and improve results today, but don't scale to arbitrarily long circuits the way true error correction will.

3 Circuit compilation: abstract to native

The gates you write are convenient abstractions — hardware speaks a smaller language.

The transpiler's job

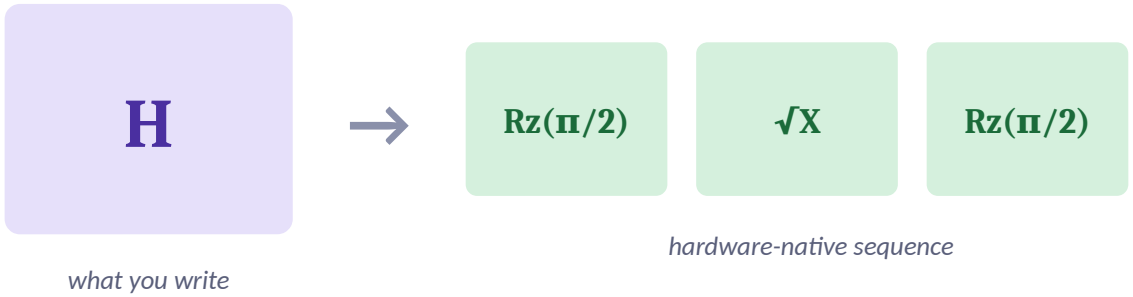
abstract gates → hardware-native gate set

Your circuit's H, CX, and Rz gates are convenient abstractions — real hardware natively supports a much smaller set, often just single-qubit rotations plus one two-qubit entangling gate.

The transpiler decomposes every gate you wrote into that native set, which can turn one logical gate into several physical pulses.

Note: two devices with different native gate sets will transpile the identical circuit into very different physical instructions — and different total error.

One gate, decomposed



One logical gate can become two or three physical pulses — each with its own small chance of error.

4 Optimization passes

A shorter circuit gives noise less time and fewer chances to act.

Making the circuit shorter, not just valid

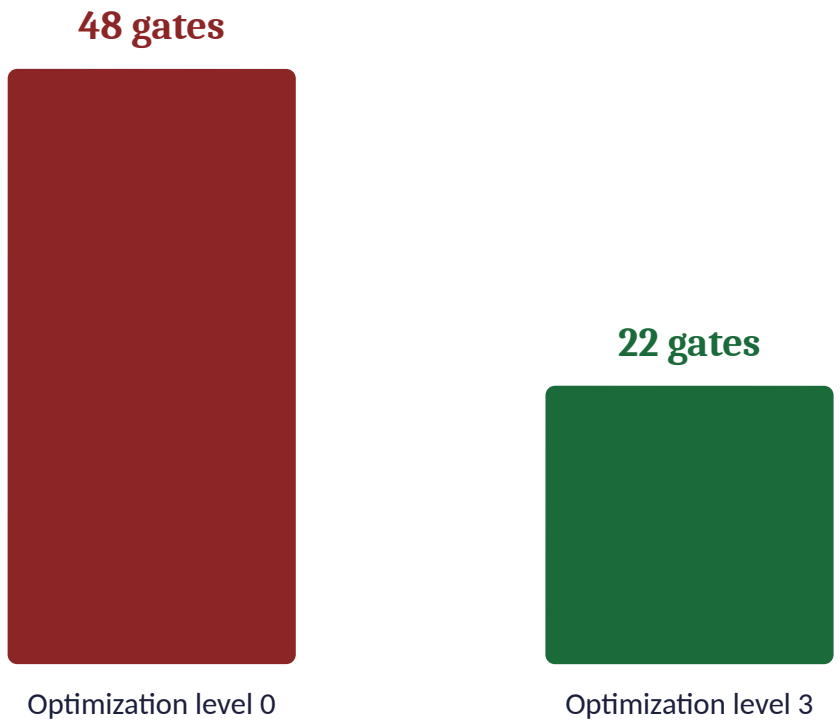
shorter circuit = less time for noise to act

Beyond satisfying the native gate set, a good transpiler actively shortens the circuit: canceling adjacent gates that undo each other, commuting gates past each other to enable further cancellation, and scheduling operations to run in parallel where possible.

Optimization level is usually a single setting (0–3 in Qiskit) trading compile time for a shorter, lower-error circuit.

Note: higher optimization levels take longer to compile but often cut circuit depth by 30–50% — usually worth it for anything beyond a quick test.

Same circuit, before and after optimization



Same logic, same result — less than half the gates, and far less exposure to noise.

5 Qubit routing & mapping

Not every qubit talks to every other — distant qubits need help to interact.

Not every qubit talks to every other

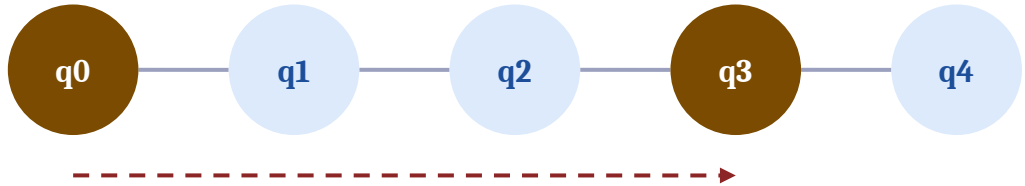
logical qubits → physical qubits + SWAP gates

Real chips connect each qubit to only a handful of neighbors. If your circuit needs a two-qubit gate between qubits that aren't adjacent, the transpiler inserts SWAP gates to move the states together first.

Each SWAP is itself built from three CX gates — so poor qubit layout can triple your two-qubit gate count before your algorithm even starts.

Note: choosing an initial qubit layout that matches your circuit's connectivity pattern is one of the highest-leverage things you can do to cut error.

A linear qubit connectivity map



q0 and q3 aren't adjacent — moving their states together costs 2 SWAPs (6 extra CX gates).

6 Zero-noise extrapolation

Deliberately adding noise to see the trend — then extrapolating it away.

Making noise worse, on purpose

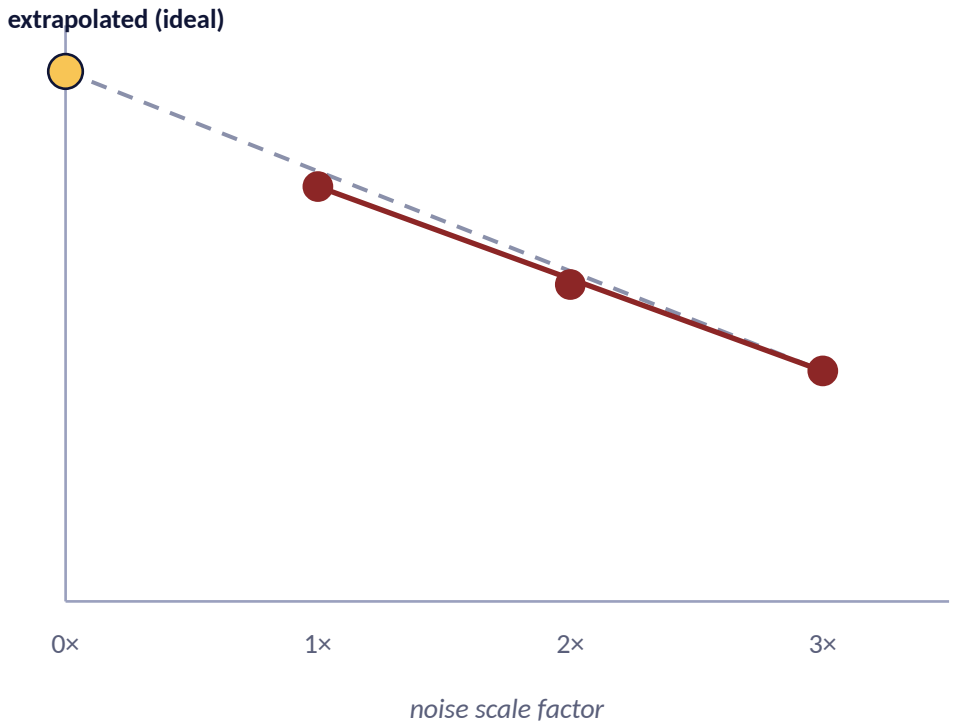
run at 1×, 2×, 3× noise → extrapolate to 0

Zero-noise extrapolation (ZNE) deliberately amplifies noise — for example by stretching gate pulses or inserting extra gate-inverse pairs — runs the circuit at several noise levels, then extrapolates the trend back to the zero-noise limit.

It needs no extra qubits, just more circuit executions — one of the most practical mitigation techniques on today's hardware.

Note: ZNE assumes the noise scales predictably; it works well for smooth, low-depth circuits, and less well as depth grows.

Expectation value vs. noise scale factor



7 Readout error mitigation

Often the single largest error source — and one of the easiest to correct after the fact.

Fixing the last step

$$\text{counts} \rightarrow \text{confusion matrix}^{-1} \rightarrow \text{corrected counts}$$

Readout errors are often the single largest error source on real hardware — a few percent chance of measuring the wrong bit value, independent of everything upstream.

By running simple calibration circuits (prepare $|0\ldots0\rangle$, prepare $|1\ldots1\rangle$, and so on) you build a confusion matrix describing how measurement mixes up outcomes, then invert it to correct your results.

Note: this correction happens entirely in post-processing — no extra quantum circuits beyond the calibration runs are needed for the algorithm itself.

1

Calibration circuits

A handful of simple runs measure how often each basis state is misread.

2

Confusion matrix

A table capturing exactly how readout errors mix up the possible outcomes.

3

Corrected counts

Inverting that matrix turns raw counts into a more accurate estimate of the truth.

8 Why it matters: mitigation vs. correction

Two different strategies, on two very different timelines.

Buying time until correction arrives

mitigation: today's tool. correction: tomorrow's requirement.

Mitigation techniques reduce bias in your results at the cost of extra circuit runs — they don't scale indefinitely, and error rates still grow with circuit size.

Full quantum error correction fixes errors during the computation itself, without needing to know the answer in advance — but needs far more physical qubits per logical qubit than mitigation ever will.

Note: most practical quantum programs today combine both: a well-compiled circuit, plus one or more mitigation techniques layered on top.

R

Mitigation scales with runs

More shots and cleverer post-processing, not more hardware.

Q

Correction scales with qubits

Needs many extra physical qubits per protected logical qubit.

\+

Today's best practice: both

Compile well, mitigate what you can, and wait for correction to arrive at scale.

ONE-LINE TAKEAWAY

A smart compiler shortens your circuit before it runs, and mitigation like zero-noise extrapolation and readout calibration squeezes a cleaner answer out of what's left — real value before full error correction arrives.

Compile smarter

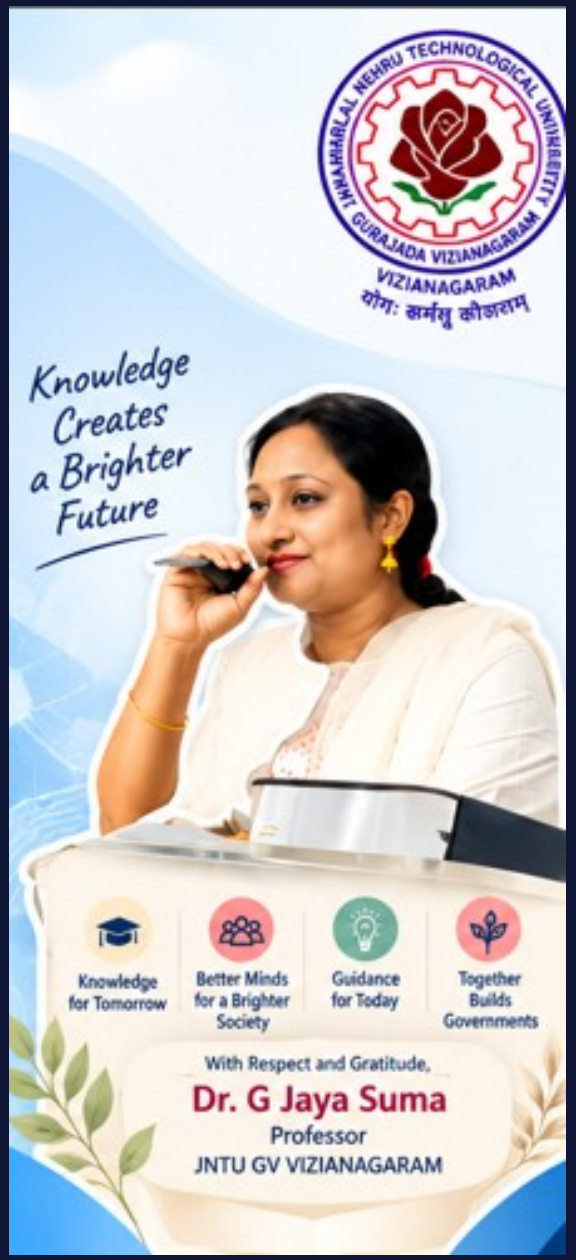
Mitigate, don't ignore

ZNE & readout calibration

→

Next: Quantum Error Correction — The Path to Fault Tolerance

We've squeezed what we can out of noisy hardware with compilation and mitigation — next we look at the long-term fix: encoding logical qubits that correct their own errors.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Quantum Error Correction Basics

How to protect fragile qubits from noise — without ever looking at the answer.

55 MIN | Bit-flip code | Phase-flip code | Stabilizers | Threshold

1 What we'll cover in 55 minutes

Seven segments, from why cloning is forbidden to the theorem that makes fault tolerance possible.

1 — 5 min

Recap: why we need error correction

Mitigation reduces bias — correction fixes errors live.

2 — 5 min

The no-cloning obstacle

Why the classical trick of copying bits can't work here.

3 — 10 min

The bit-flip code

Three qubits, majority vote, no direct measurement.

4 — 10 min

The phase-flip code

The same trick, rotated into the $|+\rangle/|-\rangle$ basis.

5 — 5 min

The Shor code: combining both

Nesting bit-flip and phase-flip protection into 9 qubits.

6 — 10 min

Stabilizers & syndrome measurement

Catching errors without ever reading the data.

7 — 10 min

Why it matters: the threshold theorem

The result that makes fault tolerance possible at all.

2 Recap: why we need error correction

Mitigation reduces bias after the fact — correction fixes errors while the computation runs.

In simple words

mitigation: reduce bias. correction: fix errors live.

We've seen decoherence erode a qubit's state, and mitigation techniques squeeze a cleaner answer out of noisy runs after the fact. Error correction goes further: it detects and fixes errors during the computation itself.

This is the only path to circuits with thousands or millions of gates — exactly what algorithms like Shor's need at cryptographic scale.

Mitigation isn't enough

!

It reduces bias in the final answer, but errors still accumulate as circuits grow.

Correction fixes errors live

✓

Detects and corrects errors during the computation, before they can spread.

The cost: many more qubits

N

Every protected ‘logical’ qubit needs several physical qubits working together.

3 The no-cloning obstacle

The classical trick of copying and voting can't work on a quantum state.

Why classical tricks don't just work

$$\text{no } U \text{ such that } U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$$

Classical error correction copies a bit three times and takes a majority vote. Quantum states can't be copied — the no-cloning theorem forbids any process that duplicates an unknown quantum state.

Quantum error correction has to protect information without ever directly copying or measuring it, since either would destroy the very superposition it's meant to preserve.

Note: the trick is to spread — not copy — one qubit's information across several entangled qubits, so errors can be detected indirectly.

No cloning allowed

You cannot duplicate an unknown qubit state, unlike a classical bit.

No direct peeking

Measuring a qubit to check it collapses the very superposition you're protecting.

The solution: spread, don't copy

Entangle the qubit's information across several physical qubits instead.

4 The bit-flip code

Three qubits, majority rule, and a correction that never looks at the data.

Encoding against bit flips

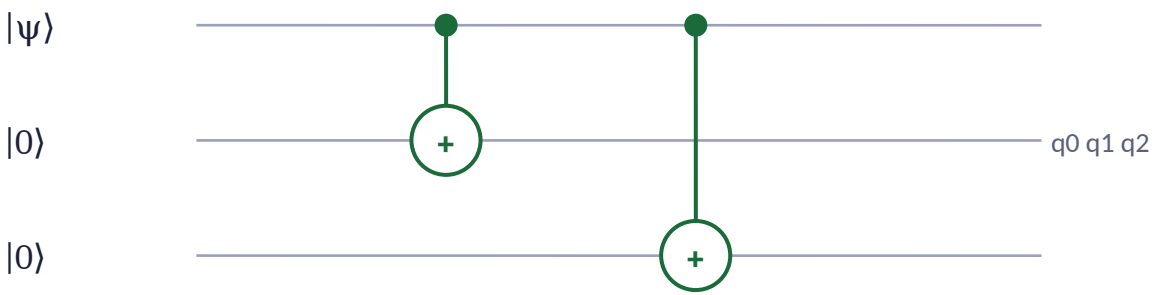
$$|0\rangle \to |000\rangle, \quad |1\rangle \to |111\rangle$$

Encode each logical qubit as three physical qubits, all agreeing: $|0\rangle$ becomes $|000\rangle$ and $|1\rangle$ becomes $|111\rangle$, with superpositions encoded the same way.

If a single bit-flip (X) error hits any one of the three qubits, comparing pairs of qubits — without measuring them directly — reveals which one flipped, and the majority still gives the right answer.

Careful: this code only protects against bit-flip (X) errors — a phase-flip (Z) error slips through completely untouched.

The encoding circuit



Single-qubit error, still correctable

000	010	100
no error → read 0	q1 flipped → read 0	q0 flipped → read 0

Majority vote recovers the right value from any single-qubit flip.

5 The phase-flip code

The same trick, rotated into the $|+\rangle/|-\rangle$ basis.

Same idea, rotated basis

$$|+\rangle \to |{+}{+}{+}\rangle, \quad |-\rangle \to |{-}{-}{-}\rangle$$

A phase-flip (Z) error is invisible in the computational basis but flips the sign in the $|+\rangle/|-\rangle$ basis. Applying Hadamards before and after turns phase flips into bit flips, so the same 3-qubit trick works — just rotated.

Encode $|+\rangle \to |{+}{+}{+}\rangle$ and $|-\rangle \to |{-}{-}{-}\rangle$, then use the same majority-style correction after transforming back.

Note: bit-flip and phase-flip codes are mirror images of each other — H·X·H = Z, so every trick for one translates directly to the other.

Bit-flip code vs. phase-flip code

	Bit-flip code	Phase-flip code
Protects against	Bit-flip (X) errors	Phase-flip (Z) errors
Encoding basis	$\|0\rangle$ / $\|1\rangle$	$\|+\rangle$ / $\|-\rangle$
Encoded states	$\|000\rangle$, $\|111\rangle$	$\|{+}{+}{+}\rangle$, $\|{-}{-}{-}\rangle$
Vulnerable to	Phase-flip (Z) errors	Bit-flip (X) errors

6 The Shor code: combining both

Nesting one code inside the other protects against any single-qubit error.

Nesting one code inside the other

3 (phase) × 3 (bit) = 9 physical qubits

Peter Shor's 1995 code nests the two: first encode against phase-flips (3 blocks), then encode each of those qubits against bit-flips (3 qubits each) — 9 physical qubits protecting one logical qubit.

This was the first code proven to protect against an arbitrary single-qubit error, not just X or Z alone — any error decomposes into a combination of the two.

Note: 9 qubits per logical qubit is expensive, but it was the proof of principle that showed full quantum error correction is possible at all.

Nested structure

Three bit-flip codes, each protected again by an outer phase-flip code layer.

Any error, corrected

Handles arbitrary single-qubit errors, not just pure bit- or phase-flips.

Not the most efficient

Later codes, like the surface code, achieve similar protection with far less qubit overhead.

7 Stabilizers & syndrome measurement

Detecting errors by checking agreement between qubits — never reading the data itself.

Detecting errors without looking at the data

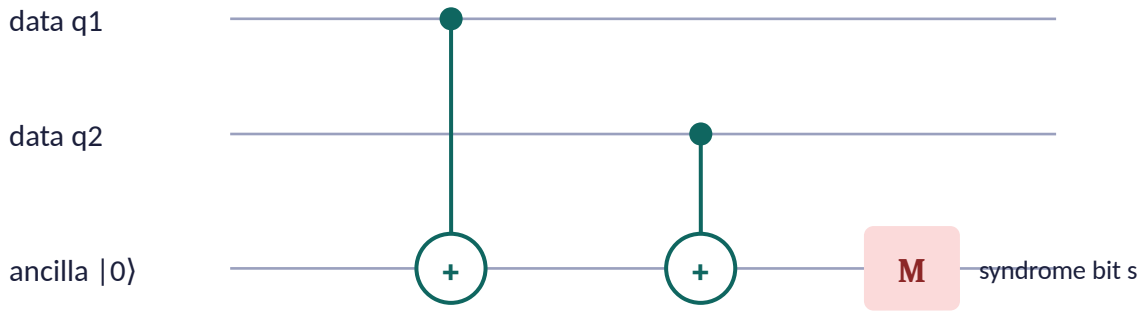
measure Z1Z2, Z2Z3 → syndrome bits, no collapse

Instead of measuring the data qubits directly, we measure joint ‘parity’ operators called stabilizers — for example, whether qubits 1 and 2 agree. This reveals whether and where an error occurred, without collapsing the encoded information.

The pattern of these outcomes is the syndrome — it points directly to the correction needed, just like a classical parity check locates an error without reading the message.

Note: stabilizer measurements use extra ‘ancilla’ qubits, so the parity check itself doesn't disturb the protected data.

A syndrome-extraction circuit



s = 0: q1 and q2 agree, no error detected here. s = 1: they disagree — an error touched this pair.

8 Why it matters: the threshold theorem

The result that makes fault-tolerant quantum computing possible in principle.

When more qubits actually helps

$$\text{error rate} < \text{threshold} \;\rightarrow\; \text{logical error} \to 0 \text{ as code grows}$$

The threshold theorem says: if your physical error rate is below some threshold (roughly 1% for surface codes), adding more physical qubits to the code makes the logical error rate exponentially smaller.

Below threshold, arbitrarily long and accurate quantum computation becomes possible in principle — above it, more qubits actually make things worse.

Note: today's best hardware sits close to, and in places just under, common threshold estimates — fault-tolerant quantum computing is a near-term research goal, not a distant dream.

✓

Below threshold: correction wins

More physical qubits per logical qubit means exponentially fewer logical errors.

✗

Above threshold: correction loses

Extra qubits just mean more error sources, with no compensating benefit.

S

The surface code

Today's leading practical candidate, needing roughly 1,000 physical qubits per reliable logical qubit.

ONE-LINE TAKEAWAY

Quantum error correction spreads one qubit's information across many, uses parity measurements to catch errors without ever peeking at the data, and — below a hardware threshold — makes reliable, arbitrarily long computation possible in principle.

Bit-flip + phase-flip = Shor code

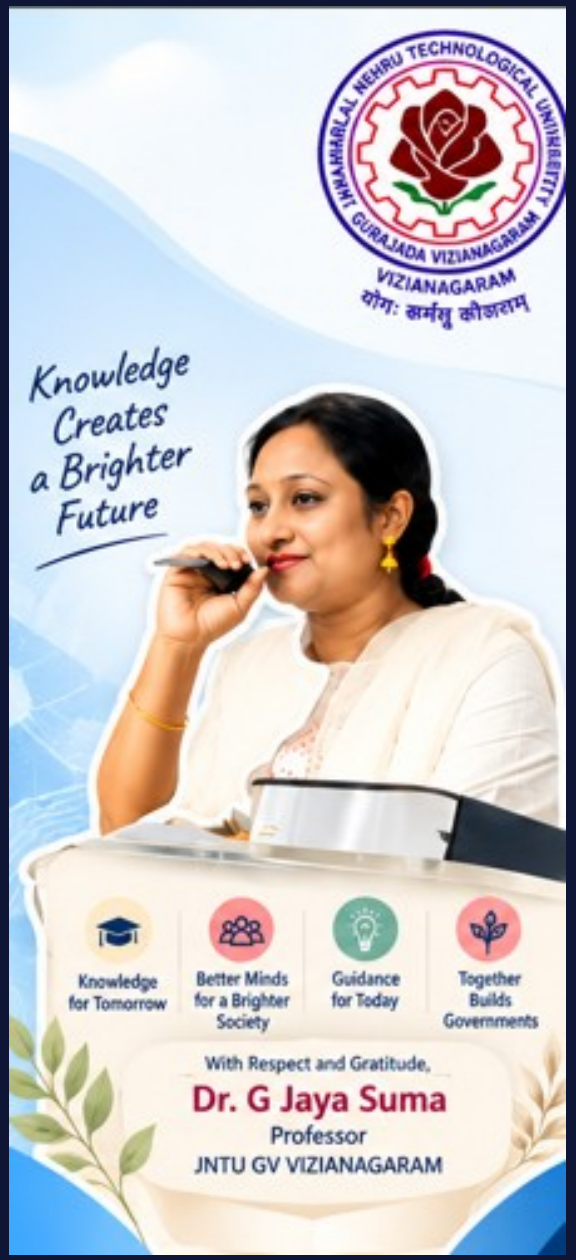
Stabilizers & syndromes

Threshold theorem

→

Next: Variational Algorithms — VQE and QAOA in Practice

Full error correction is still years away — next we go back to what today's noisy hardware can actually run, right now.



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
GURAJADA VIZIANAGARAM

TECHNOLOGY FOR A BETTER TOMORROW

Learn
Innovate
Create
Lead

VIDEO LECTURE SERIES · QUANTUM ALGORITHMS

Integrated Lab Walkthrough

One notebook, start to finish: build, mitigate, compare, and interpret.

15 MIN | Qiskit | Simulator & hardware | Mitigation | Your turn

1 What we'll cover in 15 minutes

Three short segments — from what we're building to what to try on your own.

1 — 5 min

What we're building

A 3-qubit GHZ experiment, run three different ways.

2 — 5 min

Walkthrough: circuit to corrected counts

The full build → run → mitigate → compare pipeline, in code.

3 — 5 min

Your turn

Three ways to extend the lab once the walkthrough ends.

2 What we're building

A 3-qubit GHZ experiment, run three different ways.

The lab in one sentence

GHZ state → 3 backends → 1 comparison

Today's lab strings together every piece from this series: build a 3-qubit GHZ state, run it on an ideal simulator, a noisy simulator, and (if time allows) real hardware, then use readout mitigation to see how close we can get back to the ideal answer.

Everything here reuses code you've already seen — this is about assembling the pieces, not learning new syntax.

1

Step 1: Build

The same 3-line GHZ circuit from the Hands-On Qiskit Fundamentals session.

2

Step 2: Run three ways

Ideal simulator, noisy simulator, and real hardware, if time allows.

3

Step 3: Mitigate & compare

Apply readout calibration and see how much closer you get to the ideal answer.

3 Walkthrough: circuit to corrected counts

The full pipeline, start to finish, in the same handful of lines.

The full pipeline

```
qc = ghz_circuit(3)                                    # from Segment 1

ideal     = run(qc, AerSimulator())
noisy     = run(qc, AerSimulator(noise_model=FakeSherbrooke()))
mitigated = apply_readout_correction(noisy, calibration)

compare_histograms(ideal, noisy, mitigated)
```

What each stage actually gives you

Ideal simulator

100% of outcomes land in {000, 111} — the textbook answer.

Noisy simulator

Only ~82% land in {000, 111} — noise leaks into every other outcome.

After mitigation

Back up to ~91% — closer to ideal, using only post-processing.

4 Your turn

The same pipeline, three ways to push it further.

Extending the lab

swap the circuit, the backend, or the mitigation

The exact same pipeline works for any circuit you've built this series — try swapping in Grover's algorithm, or a longer GHZ chain, and watch how the noisy-vs-mitigated gap grows with circuit size.

This is the same experiment researchers run every day to benchmark real quantum hardware.

Note: the full notebook, with solutions, is available in the course repository — there's no single 'right' way to extend it.

+

Try a bigger GHZ state

Push to 5 or 6 qubits and watch the noisy result degrade faster.

Z

Try zero-noise extrapolation

Add ZNE on top of readout mitigation and compare the improvement.

Try real hardware

Submit the same circuit to an IBM Quantum backend and compare it to the fake-backend prediction.

ONE-LINE TAKEAWAY

Build → run on multiple backends → mitigate → compare: this five-line pipeline is the same one researchers use to benchmark real quantum hardware today.

Build → Run → Mitigate

Simulator & hardware

Your lab awaits

→

Next: Keep Exploring — Build Your Own Quantum Experiments

You now have every piece — circuits, backends, mitigation, and error correction. The best next step is picking a problem you care about and building it yourself.